

Definition

Health checks in an ASP.NET Core REST API are a built-in mechanism used to **verify the operational status of an application and its dependencies**. They expose an endpoint (commonly /health) that external systems can query to determine whether the API is **healthy**, **degraded**, or **unhealthy**.

Why Health Checks Are Important

In production-grade APIs—especially **microservices**—health checks are essential for:

1. Service Availability Monitoring

- Detect if the API is running and responsive

2. Dependency Validation

- Verify connectivity to SQL Server, Redis, Azure Storage, Service Bus, etc.

3. Load Balancer Integration

- Remove unhealthy instances from traffic automatically

4. Container & Cloud Readiness

- Used by Docker, Kubernetes, Azure App Service, and Azure Kubernetes Service (AKS)

Health Check States

ASP.NET Core health checks return one of three states:

Status Meaning

Healthy	All checks passed
Degraded	API works, but one or more dependencies are slow or partially unavailable
Unhealthy	Critical dependency failure

Basic Health Check Implementation

1. Add Required Package

```
dotnet add package Microsoft.AspNetCore.Diagnostics.HealthChecks
```

2. Register Health Checks

```
builder.Services.AddHealthChecks();
```

3. Map Health Check Endpoint

```
app.MapHealthChecks("/health");
```

4. Test

GET https://localhost:5001/health

Response:

Healthy

Health Check with Database (SQL Server Example)

```
dotnet add package Microsoft.Extensions.Diagnostics.HealthChecks.SqlServer
```

```
builder.Services.AddHealthChecks()  
    .AddSqlServer(  
        builder.Configuration.GetConnectionString("DefaultConnection"),  
        name: "sqlserver");
```

Sample response:

```
{  
  "status": "Healthy",  
  "entries": {  
    "sqlserver": {  
      "status": "Healthy",  
      "duration": "00:00:00.045"  
    }  
  }  
}
```

Typical Use Cases in Real Projects

Scenario	Purpose
/health/live	Is the API process running?
/health/ready	Are all dependencies available?